

Base de donnée théorie

≡ Créateur Robin Gillard (el_pablo__)

Base de donnée → ensemble organisé de données ayant un objectif commun

SGBD (système gestion base de données) → logiciel qui va gérer les données

OLTP(online transaction processing)

- ACIDITE Transaction
- Respecte un nbr de données
- Vitesses

ACID : ensemble de propriétés qui garantissent qu'une transaction est exécutée de façon fiable

- **Atomicité** : une transaction se fait au complet ou pas du tout
- **Cohérence** : assure que chaque transaction amènera le système d'un état valide à un autre état valide
- **Isolation** : toute transaction doit s'exécuter comme si elle était la seule sur le système
- **Durabilité** : lorsqu'une transaction a été confirmée elle demeure enregistrée même à la suite d'une panne

OLAP (online analytical processing), a pas confondre avec olap ahah 😂 j'ai pas raison l'ékip → répond à des requêtes d'interrogation souvent complexe, traite de gros volumes de données et permet des ingestions massives de données

Date dans l'ordre d'apparition de tri de données :

- 1966 → IMS (information management system) (1 noeud = 1 parent)

- 1964 → IDS (integrated Data Store) (Bic on lie tout ensemble)
- 1970 → Edgar Frank Codd crée le **Modèle rationnel** (ca fait bander les informaticiens)

BD (basse de donnée) Orienté objet inconvenient :

- Cette flexibilité rend beaucoup de fonctionnalités "simples" (un tri par exemple) plus complexes à mettre en œuvre
- Les performances (pour de vrais cas d'utilisation)
- Les SGBDO ont quasi disparu

Not only SQL (NoSQL) → a la base, pour manipuler des BD géante (google, amazon, ..)

Modélisations

ER Data Model → Entity-Relationship Data Model = outil de modélisation conceptuelle

MERISE → Méthode d'analyse, de conception et de gestion de projet (informatique)

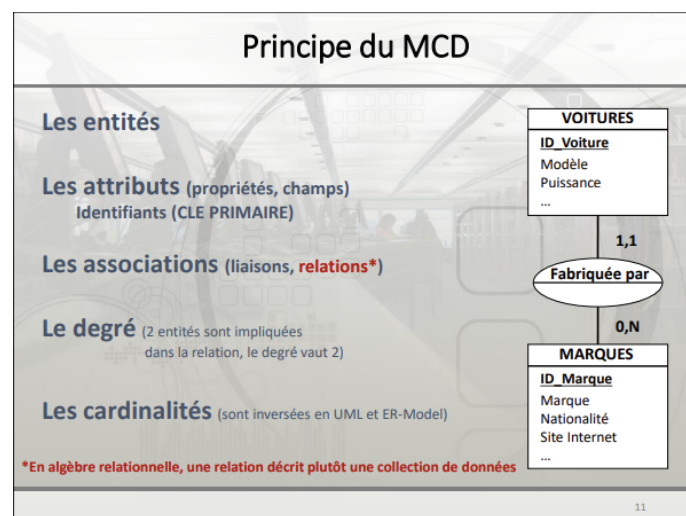
Processus de modélisation

cahier des charges → shéma conceptuel (MCD) → shéma logique (MLD) (rendre le modèle compatible avec SGBD) → shéma physique (MPD) (rendre le modèle pratique et performant avec le SGBD) → shéma base de données

MCD :

- Entités
 - regroupement d'informations communes a un "individu"

- équivalent d'une classe en POO
- schématisé par un rectangle
- Attributs
 - Caractéristiques décrivant les entités
 - stock des valeurs
 - représenté par une liste de mots explicites et concit
- Association
 - liaison logique entre une ou plusieurs entités
 - Représentée par une ellipse (en MERISE) reliée aux entités concernées
 - le nombre d'entités concernées par cette association = degré
 - Doit être affublée de cardinalités aux extrémités de l'association afin d'en caractériser le type



Cardinalité → /!\ seulement 0, 1 ou N

Clé primaire :

- Propriété naturelle → être certain que la valeur soit unique, éviter les modifications
 - Personne → email

- Propriété artificielle → performance supérieur (le go to si tu sais pas quoi faire)
 - Client → ID_client
- Propriété composée
 - Entreprise → Enseigne + localité

▼ Explication clé IA

Clé primaire naturelle

Une clé primaire naturelle est basée sur une propriété existante et significative des données.

Avantages:

- Utilise des données déjà présentes, pas besoin d'ajouter un champ supplémentaire
- A une signification métier

Inconvénients:

- Risque de modification au fil du temps
- Peut être complexe si composée de plusieurs champs

Exemple:

Utiliser l'email comme clé primaire pour une table Personne.

- L'email est généralement unique pour chaque personne
- Mais attention, un email peut changer si la personne change d'adresse

Clé primaire artificielle

Une clé primaire artificielle est un identifiant créé spécifiquement, souvent auto-incrémenté.

Avantages:

- Simple et performant
- Ne change jamais
- Occupe peu d'espace (souvent un entier)

Inconvénients:

- N'a pas de signification métier
- Nécessite un champ supplémentaire

Exemple:

Utiliser un ID_client auto-incrémenté comme clé primaire pour une table Client.

- Chaque nouveau client reçoit automatiquement un ID unique
- Très efficace pour les jointures et indexations

Clé primaire composée

Une clé primaire composée utilise plusieurs champs combinés pour assurer l'unicité.

Avantages:

- Peut refléter une contrainte métier complexe
- Utilise des données existantes

Inconvénients:

- Plus complexe à gérer
- Performances potentiellement réduites pour les jointures

Exemple:

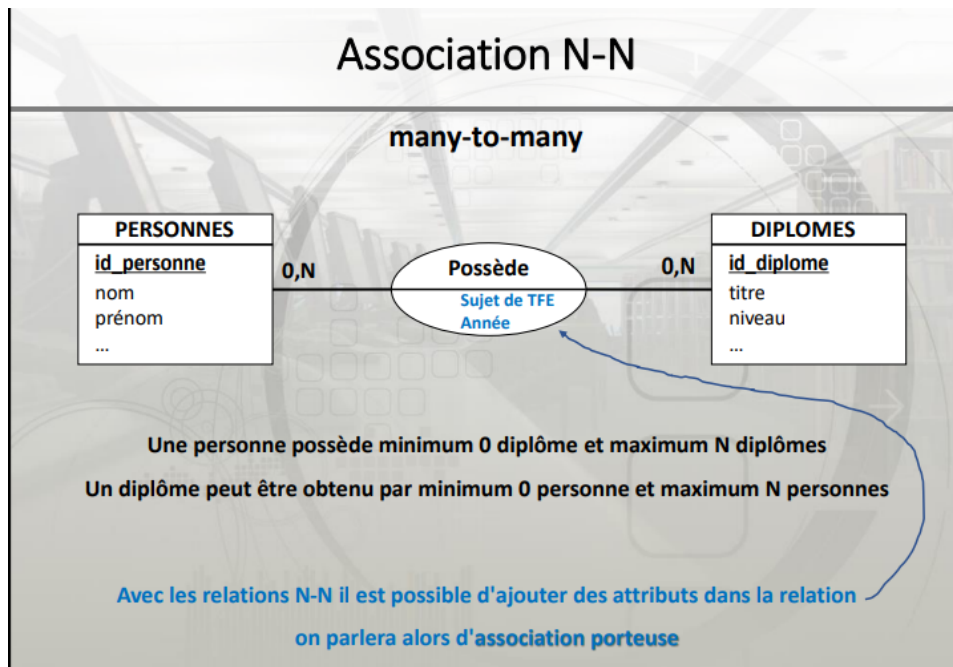
Utiliser la combinaison Enseigne + Localité comme clé primaire pour une table Entreprise.

- Permet de différencier des franchises ayant le même nom dans différentes villes
- Mais attention si une entreprise change de nom ou déménage

En résumé, le choix de la clé primaire dépend du contexte :

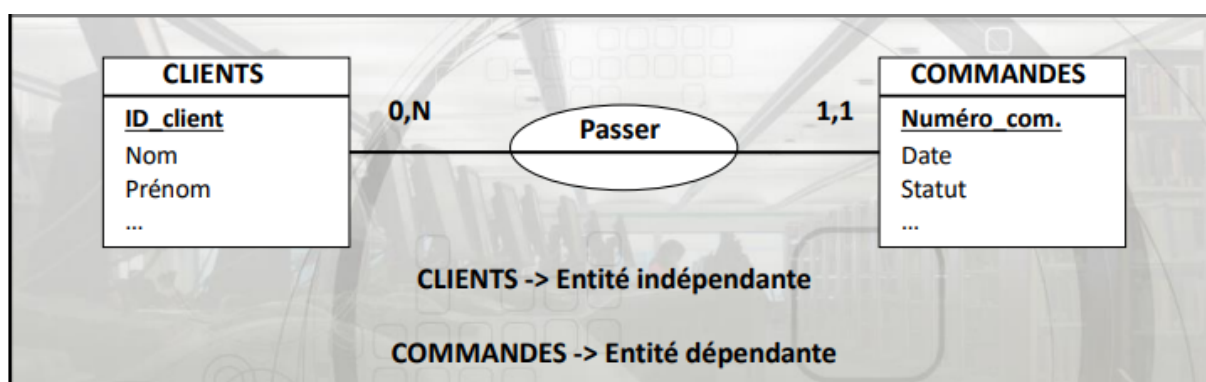
- Une clé naturelle si elle est vraiment stable et significative
- Une clé artificielle pour la simplicité et les performances
- Une clé composée si nécessaire pour garantir l'unicité, mais à utiliser avec précaution

Le plus souvent, une clé artificielle de type ID auto-incrémenté est le choix le plus sûr et le plus simple à long terme



entité indépendante on écrira 0,N

entité dépendante on écrira 1,1 la cardinalité

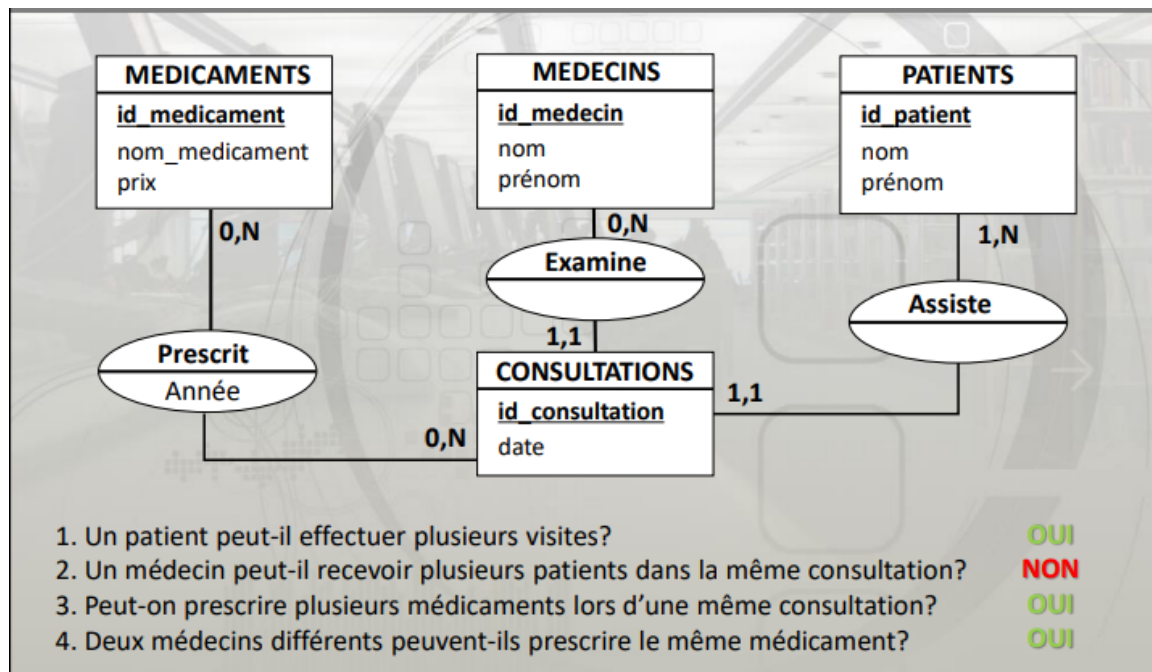


Association plurielle → 1 entité (classe) qui a plusieurs liens

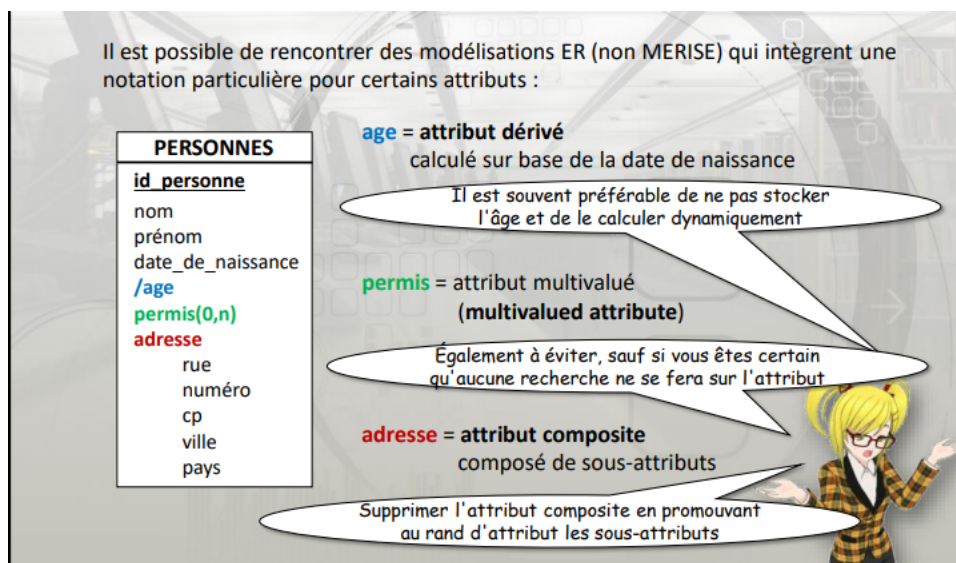
association reflexive → revient vers soi

Association ternaire → 3 entité (classe)

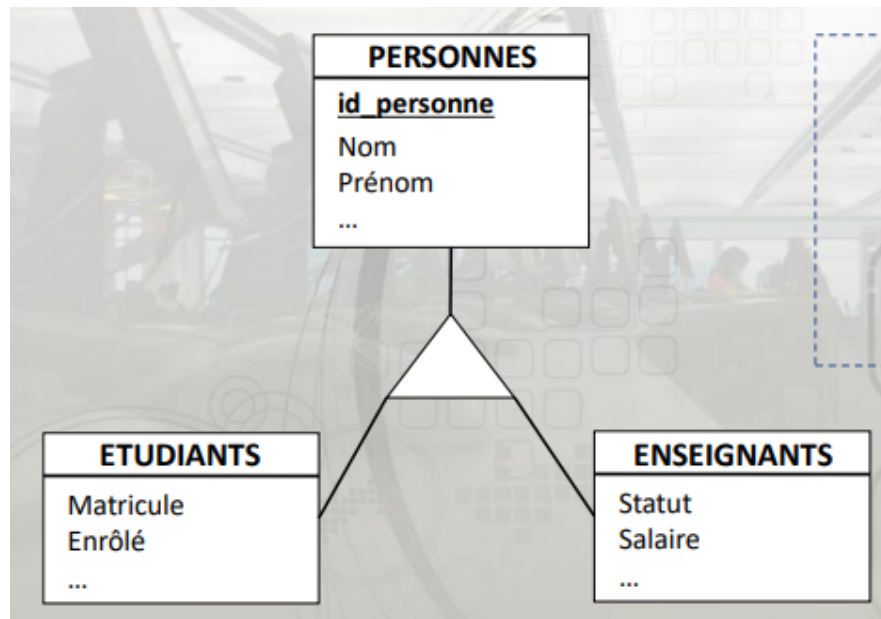
▼ exo



Attribut spéciaux :



▼ Lien de généralisation :



Ressemble a l'héritage en poo

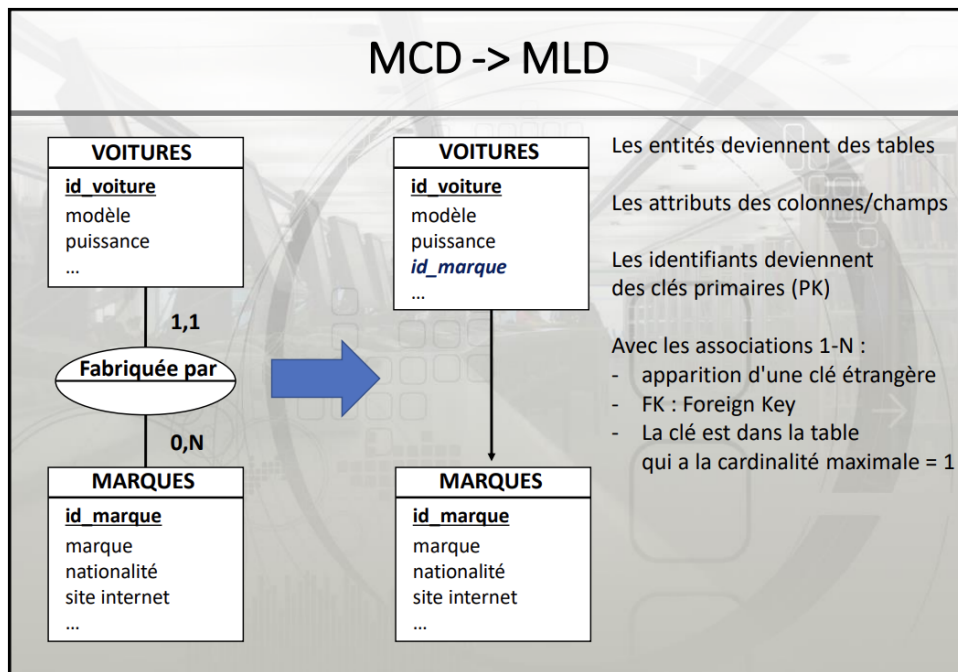
Si on a un + dans le triangle → Une personne est soit un étudiant soit un enseignant

Si on a un X dans le triangle → Une personne est soit un étudiant soit un enseignant soit aucun des deux

Si on a un T dans le triangle → Une personne est soit un étudiant soit un enseignant ou bien les 2 en même temps

Si le triangle ressemble a l'image suivante, Une personne est soit un étudiant soit un enseignant soit aucun des 2, ou bien les 2 en même temps





on ajoute une flèche de l'endroit où la foreign key est apparue vers la d'où elle vient

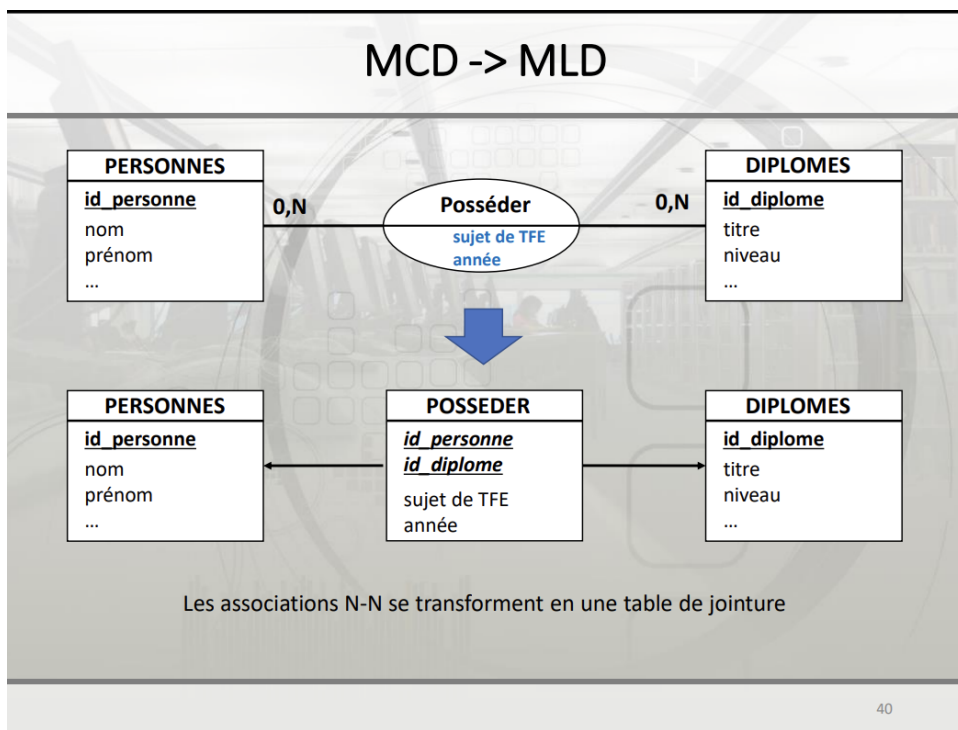
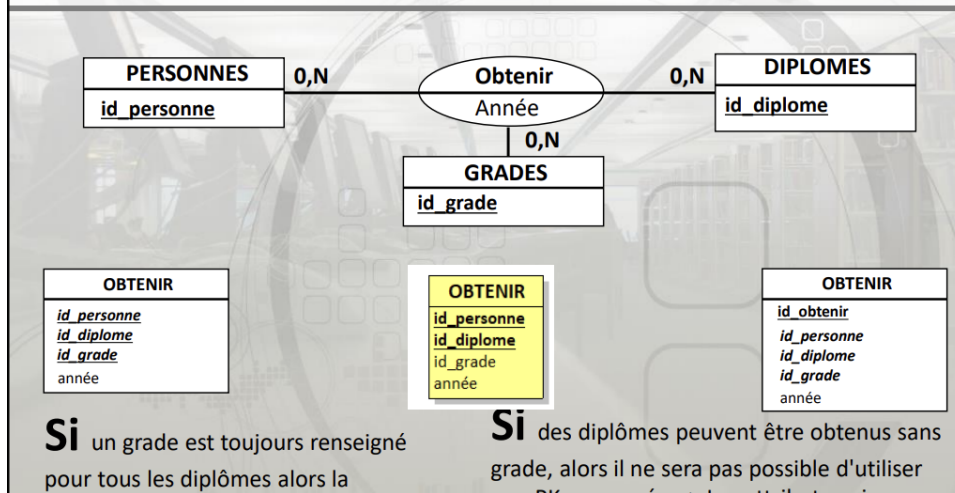
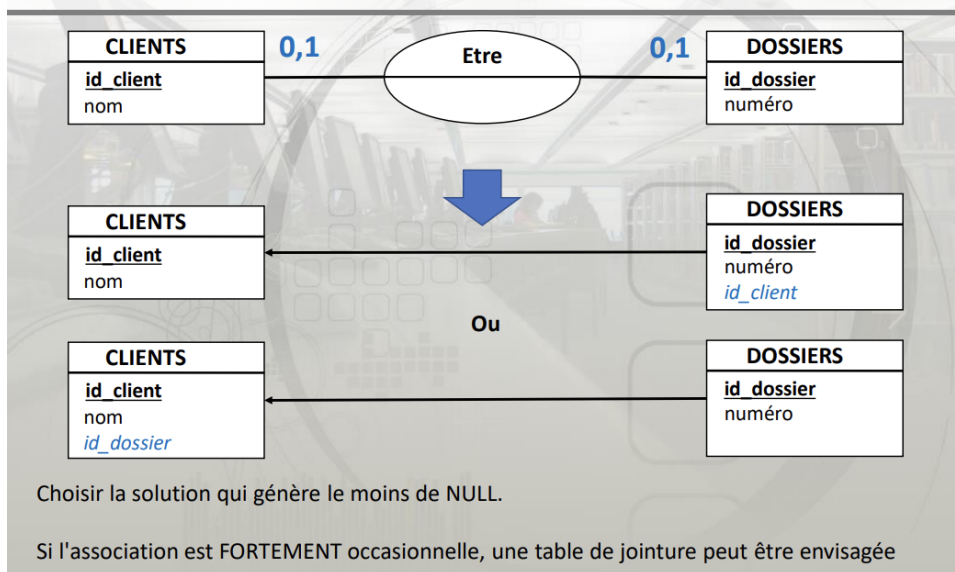


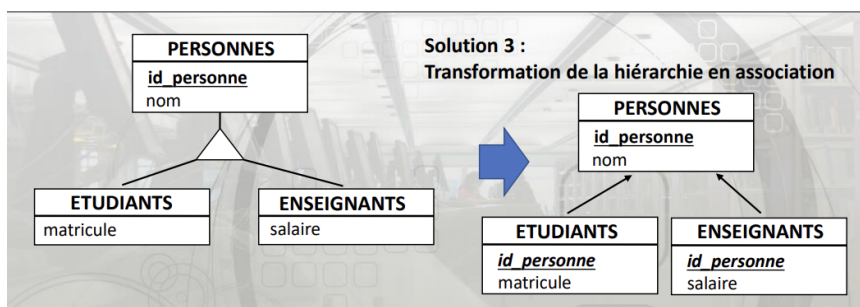
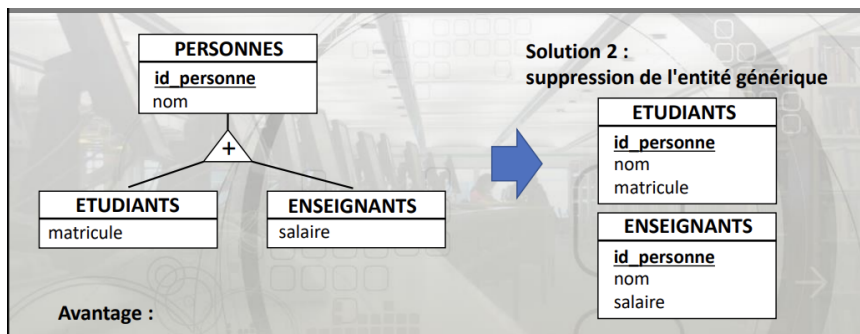
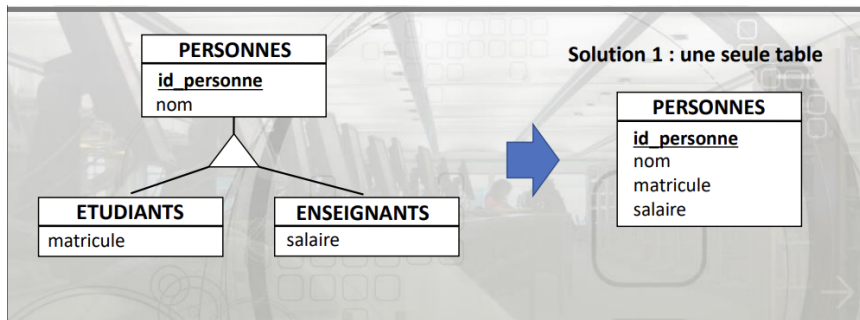
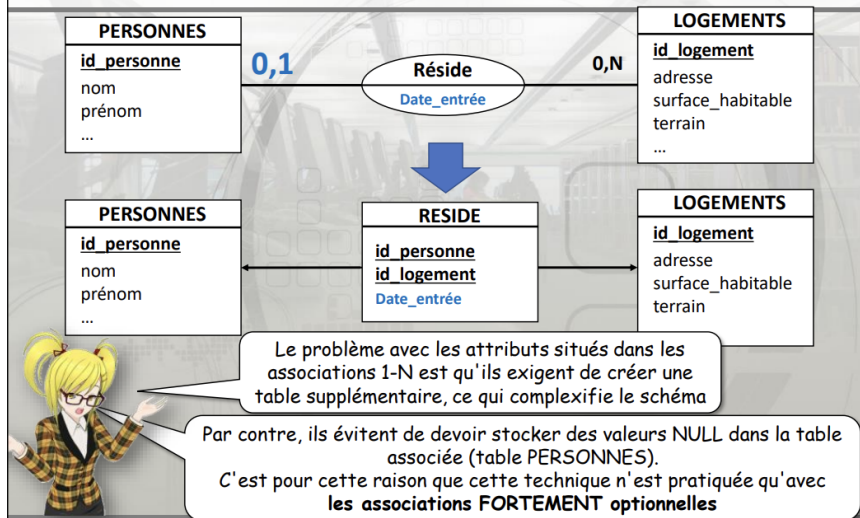
Table de jointure et PK

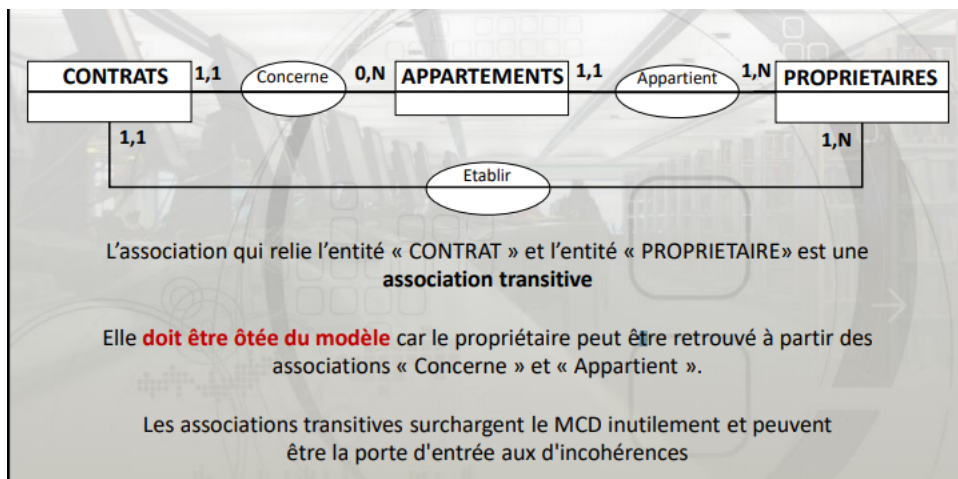
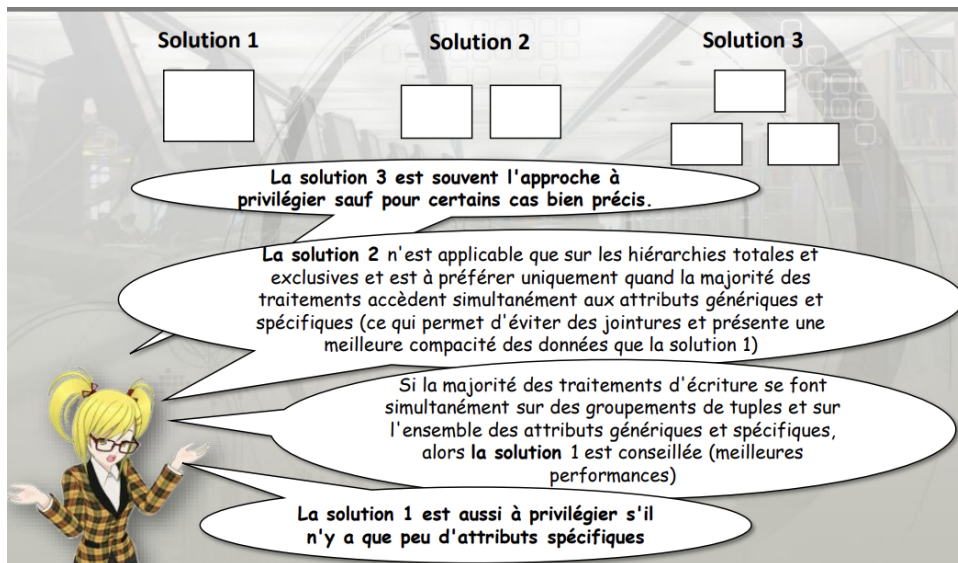


MCD -> MLD : associations 1 - 1



Attribut dans une association 1-N





▼ Normalisation image

Première forme normale : 1FN

id	nom	email
1	M. Frank Dewit	f.dewit@gmail.com, zedewit@hotmail.com
2	M. Ivan Raoul	rivan@hotmail.fr
3	Mme Leen Breckx	Leen.breckx@gmail.com, brekyleen@live.fr

Les attributs doivent être atomiques :

- Supprimer les attributs composites en promouvant au rang d'attribut les sous-attributs
- Les attributs multivalués donnent naissance à de nouvelles entités

id	titre	prenom	nom
1	M.	Frank	Dewit
2	M.	Ivan	Raoul
3	Mme	Leen	Breckx

0,N

1,1

id	email
1	f.dewit@gmail.com
2	zedewit@hotmail.com
3	rivan@hotmail.fr
4	Leen.breckx@gmail.com
5	brekyleen@live.fr

Deuxième forme normale : 2FN

Respecter la 1FN

Les attributs doivent dépendre complètement de la clé et non partiellement

Ce cas se rencontre principalement avec les clés composées.

Exemple avec une entité qui contient les évaluations des films par des utilisateurs :

id_film	id_user	note	genre
1	1	3	Comédie
2	1	5	Action
2	8	4	Action
3	8	1	Aventure

Pas en 2FN !

L'attribut "genre" est propre au film (id_film) et ne dépend pas de l'utilisateur (id_user), cet attribut n'est pas dans la bonne entité.

Troisième forme normale : 3FN

Respecter la 2FN

Pas de dépendance fonctionnelle entre les attributs non clé

Exemple avec une entité qui contient les évaluations des films par des utilisateurs :

id_film	id_user	note	image_note
1	1	3	3etoiles.png
2	1	5	5etoiles.png
2	8	4	4etoiles.png
3	8	1	1etoiles.png

Pas en 3FN !

id_film	id_user	note
1	1	3
2	1	5
2	8	4
3	8	1

1,1 0,N

note	image_note
3	3etoiles.png
5	5etoiles.png
4	4etoiles.png
1	1etoiles.png

Forme normale de Boyce-Codd : BCNF

Respecter la 3FN

Les attributs non clé ne doivent pas être source de dépendance fonctionnelle vers une partie de la clé

Cela peut être interprété comme la contrainte inverse de la 2FN.

Exemple avec une entité contenant un historique des transports :

id_pays	id_user	date	transport	ville
1	1	10/10/2018	Voiture	Bruxelles
1	2	11/10/2018	Bus	Liège
3	3	11/10/2018	Avion	Boston
4	2	11/11/2018	Voiture	Paris

Dépendances fonctionnelles :

id_pays, id_user -> date OK
id_pays, id_user -> transport OK
ville -> id_pays **Pas en BCNF**

Quatrième forme normale : 4FN

Respecter la BCNF

Une relation est en 4FN si pour chaque dépendance multivaluée $X \twoheadrightarrow Y$ non triviale, X est une superclé de la relation

...

Exemple avec l'entité "livres" :

Les "formes normales" font partie de ces choses très complexes à formuler mais qui sont plutôt simples à mettre en œuvre

isbn	auteur	mot_clé
1-12312-323-2	Marc	Tortue
1-12312-323-2	Michel	Tortue
1-12312-323-2	Marc	Natation
1-12312-323-2	Michel	Natation
8-12312-323-4	Alice	Hiver

Pas en 4FN !



isbn	auteur
1-12312-323-2	Marc
1-12312-323-2	Michel
8-12312-323-4	Alice

isbn	mot_clé
1-12312-323-2	Tortue
1-12312-323-2	Natation
8-12312-323-4	Hiver

▼ Normalisation Texte

La normalisation est un processus essentiel en modélisation de bases de données. Elle vise à organiser les données pour minimiser la redondance et éviter les anomalies de mise à jour.

Objectifs de la normalisation :

- **Améliorer la modélisation** : Assurer que les structures de données sont logiques et efficaces.
- **Éviter les redondances** : Réduire la duplication des données pour éviter les incohérences.
- **Éviter les anomalies** : Protéger contre les problèmes lors des insertions, mises à jour ou suppressions.

Niveaux de normalisation :

1. Première Forme Normale (1NF) :

- Les attributs doivent être atomiques (pas de listes ou ensembles).
- Exemple : Si une table contient plusieurs emails pour un utilisateur, il faut créer une nouvelle table pour les emails.

2. Deuxième Forme Normale (2NF) :

- Respecte la 1NF.

- Les attributs doivent dépendre entièrement de la clé primaire, pas partiellement.
- Exemple : Dans une table d'évaluations de films, le genre du film ne doit pas dépendre de l'utilisateur mais uniquement du film.

3. Troisième Forme Normale (3NF) :

- Respecte la 2NF.
- Aucune dépendance fonctionnelle entre attributs non clés.
- Exemple : Si une image de note dépend d'une note, il faut séparer ces informations dans deux tables.

4. Forme Normale de Boyce-Codd (BCNF) :

- Respecte la 3NF.
- Les attributs non clés ne doivent pas déterminer une partie de la clé primaire.
- Exemple : Si une ville détermine un pays, cette relation doit être isolée.

5. Quatrième Forme Normale (4NF) :

- Respecte la BCNF.
- Élimine les dépendances multivaluées non triviales.
- Exemple : Si un livre a plusieurs auteurs et mots-clés, il faut créer des tables séparées pour chaque relation.

La normalisation est principalement appliquée dans les systèmes OLTP (Online Transaction Processing) pour garantir l'intégrité des données et optimiser les transactions. Elle peut être ajustée ou partiellement dénormalisée pour améliorer les performances dans certains contextes.

SQL

SQL (structured Query Language) → langage de prog déclaratif

Crée en 1982

Les tables ne sont pas naturellement ordonnées

4 Façon pour intégrer le SQL dans un logiciel :

▼ **Utilisation d'un connecteur de BD** avec écriture des requêtes sous forme de chaînes de caractères.

Le connecteur se connecte :

- soit à un driver dédié à un SGBD particulier
- soit à un middleware plus générique (par exemple : ODBC) qui permet de se connecter à son tour à un driver dédié

Les requêtes sont écrites sous forme de chaînes de caractères envoyées comme paramètre de fonction/méthode au connecteur

▼ **Embedded SQL** : intégration du langage SQL directement dans le code source de l'application.

Intégration du langage SQL directement dans le code source de l'application.

Dans ce cas, le SQL est analysé par le compilateur/interpréteur de l'application

▼ **Utilisation d'un ORM (Object Relational Mapping)** qui permet de générer automatiquement le SQL via une couche d'abstraction Objet

Couche logicielle entre l'application et le SGBDR permettant d'accéder à la base de

données et son contenu en utilisant des objets.

Le SQL est généré automatiquement

Hibernate, Entity Framework et Doctrine sont les plus connus

▼ **Principe des bases de données épaisses** : le SQL n'est pas dans l'application mais dans le SGBDR (utilisation de procédures stockées)

La BD est accédée uniquement par des :

- Vues : une table virtuelle définie par une requête SQL stockée côté serveur
- Procédures Stockées : fonctions précompilées et stockées côté serveur. Elles peuvent contenir du SQL ainsi que du code impératif (procédural)

Introduction au SQL

SQL (Structured Query Language) est le langage standard pour interagir avec les bases de données relationnelles. Ses principales caractéristiques sont :

- Langage déclaratif ressemblant à des phrases en anglais
- Mathématiquement complet depuis SQL:1999
- Divisé en plusieurs sous-langages : DDL, DML, DCL, TCL, SQL/PSM

SQL peut être utilisé de différentes manières dans les applications :

1. Via des connecteurs de base de données (ex: JDBC en Java)
2. Embedded SQL intégré directement dans le code source
3. ORM (Object-Relational Mapping) qui génère le SQL automatiquement
4. Procédures stockées dans la base de données

Data Definition Language (DDL)

Le DDL permet de définir et modifier la structure de la base de données :

```
CREATE DATABASE ma_bd;
CREATE TABLE utilisateurs (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE
);
ALTER TABLE utilisateurs ADD COLUMN date_naissance DATE;
DROP TABLE vieille_table;
```

Si vous avez oublié quelque chose → ALTER TABLE

Les principaux types de données en SQL incluent :

- Texte : CHAR, VARCHAR, TEXT
- Numérique : INT, BIGINT, DECIMAL, FLOAT
- Date/Heure : DATE (année mois jour), TIME, DATETIME(same date + heure minute seconde), TIMESTAMP(temps écoulé depuis 1janvier 1970)
- Booléen : BOOLEAN
- Binaire : BLOB

L'interclassement est une table de correspondance spécifique à un jeu de caractères qui permet l'ordonnement correct d'une liste de caractères. Il définit les règles de comparaison et de tri des caractères dans une base de données.

Caractéristiques principales

- Un jeu de caractères peut avoir plusieurs interclassements
- Il détermine l'ordre des caractères (par exemple, "é" après "e")
- Il établit la hiérarchie entre majuscules et minuscules
- Il définit l'équivalence entre caractères (comme "e", "E", "é", "è", "ê")

Data Manipulation Language (DML)

Le DML permet de manipuler les données dans les tables :

INSERT

```
INSERT INTO utilisateurs (nom, email)
VALUES ('Alice', 'alice@email.com');

INSERT INTO utilisateurs (nom, email)
VALUES
    ('Bob', 'bob@email.com'),
    ('Charlie', 'charlie@email.com');
```

num_chassis	voiture	couleur
abcdabcde13	Polo	grise
yzzzABC4442	Golf	rouge
BKxyOMzz13	Clio	bleue
xxxybbzzz14	Clio	jaune
zaeaFJW123	Punto	rouge

La requête suivante va provoquer une erreur (PK dupliquée) :

```
INSERT INTO ma_table VALUES ( 'xxxybbzzz14','Clio','noire');
```

Il est possible d'ignorer l'erreur et rien ne sera ajouté dans la table

```
INSERT IGNORE INTO ma_table VALUES ( 'xxxybbzzz14','Clio','noire');
```

Il est possible de transformer l'insertion en une modification en cas de PK dupliquée :

```
INSERT INTO ma_table VALUES ( 'xxxybbzzz14','Clio','noire')
ON DUPLICATE KEY UPDATE voiture='Clio', couleur='noire';
```

UPDATE

```
UPDATE utilisateurs
SET email = 'nouvelle@email.com'
WHERE id = 1;
```

Attention à ne pas oublier la clause WHERE sinon tous les enregistrements de la table seront modifiés (sauf si c'était l'effet souhaité).

DELETE

```
DELETE FROM utilisateurs WHERE id = 1;
```

Attention à ne pas oublier le WHERE, sinon la table sera vidée : DELETE FROM ma_table;

DELETE = DML

TRUNCATE = DDL

DELETE et TRUNCATE sont très différents. Contrairement à DELETE, TRUNCATE :

- ne déclenche pas les triggers (déclencheurs)
- ne peut pas être exécuté sur les tables munies de contraintes d'intégrité référentielle (gestion des clés étrangères liées à la table)

- réinitialise les compteurs d'auto-incrémentation

SELECT

La clause SELECT est la plus complexe et puissante du SQL :

```
SELECT nom, email
FROM utilisateurs
WHERE date_inscription > '2023-01-01'
ORDER BY nom ASC
LIMIT 10;
```

Principales clauses :

- FROM : spécifie les tables sources
- WHERE : filtre les lignes (= < > ≥ ≤ <> like, between, and, or, not, in)
- GROUP BY : regroupe les résultats
- HAVING : filtre les groupes
- ORDER BY : trie les résultats (asc, desc, field (on choisit l'ordre des étapes))
- LIMIT : limite le nombre de résultats

Opérateurs et fonctions utiles :

- DISTINCT : élimine les doublons
- LIKE : recherche par motif (ex: 'A%' pour les noms commençant par A)
- BETWEEN : range de valeurs
- IN : liste de valeurs
- Agrégation : COUNT(), SUM(), AVG(), MIN(), MAX()
- Chaînes : CONCAT()(permet de concaténer (joindre) deux ou plusieurs chaînes de caractères en une seule.), SUBSTRING()(extrait une partie d'une chaîne de caractères. tel que :

`SUBSTRING(string, start, length)` string : la chaîne source start : position de départ (commence généralement à 1) length : nombre de caractères à extraire)

UPPER(), LOWER()

- Date : YEAR(), MONTH(), DATEDIFF()

Exemples avancés :

```
-- Sous-requête
SELECT nom
FROM utilisateurs
WHERE id IN (SELECT utilisateur_id FROM commandes);

-- Jointure
SELECT u.nom, c.montant
FROM utilisateurs u
JOIN commandes c ON u.id = c.utilisateur_id;

-- Agrégation avec groupement
SELECT pays, COUNT(*) as nb_utilisateurs
FROM utilisateurs
GROUP BY pays
HAVING nb_utilisateurs > 100
ORDER BY nb_utilisateurs DESC;
```

Types de jointures

Les jointures permettent de combiner des lignes de deux ou plusieurs tables en fonction d'une condition de jointure.

1. INNER JOIN

L'INNER JOIN retourne uniquement les lignes qui ont des correspondances dans les deux tables.

Exemple:

```
SELECT employees.name, departments.department_name
FROM employees
INNER JOIN departments ON employees.department_id = departments.id;
```

Cela retournera uniquement les employés qui ont un département associé.

2. LEFT JOIN (ou LEFT OUTER JOIN)

Le LEFT JOIN retourne toutes les lignes de la table de gauche (première table), et les lignes correspondantes de la table de droite. S'il n'y a pas de correspondance, le résultat est NULL du côté droit.

Exemple:

```
SELECT employees.name, departments.department_name
FROM employees
LEFT JOIN departments ON employees.department_id = departments.id;
```

Cela retournera tous les employés, même ceux qui n'ont pas de département associé.

3. RIGHT JOIN (ou RIGHT OUTER JOIN)

Le RIGHT JOIN est l'inverse du LEFT JOIN. Il retourne toutes les lignes de la table de droite, et les lignes correspondantes de la table de gauche.

Exemple:

```
SELECT employees.name, departments.department_name
FROM employees
RIGHT JOIN departments ON employees.department_id = departments.id;
```

Cela retournera tous les départements, même ceux qui n'ont pas d'employés.

4. FULL JOIN (ou FULL OUTER JOIN)

Le FULL JOIN retourne toutes les lignes lorsqu'il y a une correspondance dans l'une ou l'autre des tables.

Exemple:

```
SELECT employees.name, departments.department_name
FROM employees
FULL JOIN departments ON employees.department_id = departments.id;
```

Cela retournera tous les employés et tous les départements, avec des NULL là où il n'y a pas de correspondance.

Opérateurs ensemblistes

Les opérateurs ensemblistes permettent de combiner les résultats de deux ou plusieurs requêtes SELECT.

1. UNION

UNION combine les résultats de deux ou plusieurs requêtes SELECT en éliminant les doublons.

Exemple:

```
SELECT name FROM employees
UNION
SELECT name FROM customers;
```

Cela retournera une liste unique de tous les noms d'employés et de clients.

2. UNION ALL

UNION ALL est similaire à UNION, mais il n'élimine pas les doublons.

Exemple:

```
SELECT name FROM employees
UNION ALL
SELECT name FROM customers;
```

Cela retournera tous les noms, y compris les doublons.

3. INTERSECT

INTERSECT retourne uniquement les lignes qui sont présentes dans les résultats des deux requêtes.

Exemple:

```
SELECT city FROM employees
INTERSECT
SELECT city FROM customers;
```

Cela retournera les villes qui ont à la fois des employés et des clients.

4. EXCEPT (ou MINUS)

EXCEPT retourne les lignes de la première requête qui ne sont pas présentes dans le résultat de la seconde requête.

Exemple:

```
SELECT city FROM employees  
EXCEPT  
SELECT city FROM customers;
```

Cela retournera les villes qui ont des employés mais pas de clients.

POUR LES JOINTURE → P74 PDF

Exemple :

```
SELECT u.nom, c.date, c.montant  
FROM utilisateurs u  
LEFT JOIN commandes c ON u.id = c.utilisateur_id;
```

Types de données supplémentaires

- ENUM : permet de définir une liste de valeurs autorisées pour une colonne
- SET : similaire à ENUM mais permet de sélectionner plusieurs valeurs
- JSON : pour stocker des données au format JSON (disponible dans certains SGBD)

Contraintes d'intégrité

- PRIMARY KEY : définit la clé primaire d'une table
- FOREIGN KEY : définit une clé étrangère faisant référence à une autre table
- UNIQUE : assure qu'une colonne ou un groupe de colonnes contient des valeurs uniques

- CHECK : permet de définir une condition que les données doivent respecter

Opérations ensemblistes

SQL permet d'effectuer des opérations ensemblistes sur les résultats de requêtes :

- UNION : combine les résultats de deux requêtes en éliminant les doublons
- UNION ALL : combine les résultats sans éliminer les doublons
- INTERSECT : retourne les lignes communes aux deux requêtes
- EXCEPT : retourne les lignes de la première requête qui ne sont pas dans la seconde

Gestion des valeurs NULL

- IS NULL : teste si une valeur est NULL
- IS NOT NULL : teste si une valeur n'est pas NULL
- COALESCE : retourne la première valeur non NULL d'une liste

Requêtes imbriquées :

1. Requêtes imbriquées 1-1

Ces requêtes retournent une seule colonne et une seule ligne (une seule valeur).

Requêtes imbriquées : exemple 1-1

id_type	nom_type	id	id_type	nom	prix
1	Sandwich	1	1	Poulet-curry	2.50
2	Burger	2	2	Royal-Cheese	5.10
3	Pizza	3	1	Martino	3.00
4	Salade	4	2	King-Bacon	5.40
		5	3	4 fromages	8.50

Table *types_plat*

Table *plats*

```

SELECT
  t.nom_type,
  (SELECT MIN(p1.prix) FROM plats AS p1 WHERE p1.id_type=t.id_type) AS 'min',
  (SELECT MAX(p2.prix) FROM plats AS p2 WHERE p2.id_type=t.id_type) AS 'max'
FROM types_plat AS t
ORDER BY t.nom_type

```

type_plat	min	max
Burger	5.10	5.40
Pizza	8.50	8.50
Salade	NULL	NULL
Sandwich	2.50	3.00

Avec ce type d'utilisation, une requête imbriquée ne peut renvoyer qu'une seule valeur.

Les requêtes ne renvoyant qu'une seule valeur peuvent être imbriquées presque partout

2. Requêtes imbriquées N-1

Ces requêtes retournent plusieurs colonnes mais une seule ligne.

Requêtes imbriquées : exemple N - 1

id	nom
1	Sandwich
2	Burger
3	Pizza
4	Salade
5	Lasagne

Table *plats*

id	menu	prix
1	Sandwich	4.00
2	Burger	7.30
3	Pizza	11.50
4	Mitraillette	5.50
5	Salade	6.00

Table *menus*

id	nom
1	Sandwich
2	Burger
3	Pizza

```

SELECT p.id, p.nom FROM plat AS p
WHERE (p.id, p.nom) MATCH (SELECT m.id, m.menu FROM menus AS m WHERE m.id=p.id)
ORDER BY p.id

```

Actuellement, l'opérateur MATCH n'est pas fonctionnel avec MySQL/MariaDB

3. Requêtes imbriquées 1-N

Ces requêtes retournent une seule colonne mais plusieurs lignes.

Requêtes imbriquées : exemple 1 - N

Table *types_plat*

id_type	nom
1	Sandwich
2	Burger
3	Pizza

Table *plats*

id	id_type	nom	prix
1	1	Poulet-curry	2.50
2	2	Royal-Cheese	5.10
3	1	Martino	3.00
4	2	King-Bacon	5.40
5	3	4 fromages	8.50

```
SELECT DISTINCT nom FROM types_plat
WHERE id_type IN (
    SELECT id_type FROM plats
    WHERE (prix BETWEEN 3 AND 6)
);
```

nom
Burger
Sandwich

4. Requêtes imbriquées N-N

Ces requêtes retournent plusieurs colonnes et plusieurs lignes, essentiellement une table complète.

Requêtes imbriquées : exemple N - N

id	nom
1	Sandwich
2	Burger
3	Pizza
4	Salade
5	Lasagne

Table *plats*

id	nom
1	Sandwich
2	Burger
3	Pizza

```
SELECT p.id, p.nom FROM
    (SELECT id, nom FROM plats WHERE id<4) AS p
ORDER BY p.id
```

Chapitre 6 : fonction avancées

Avantages des index :

- Accélèrent (considérablement) les recherches, jointures et agrégations

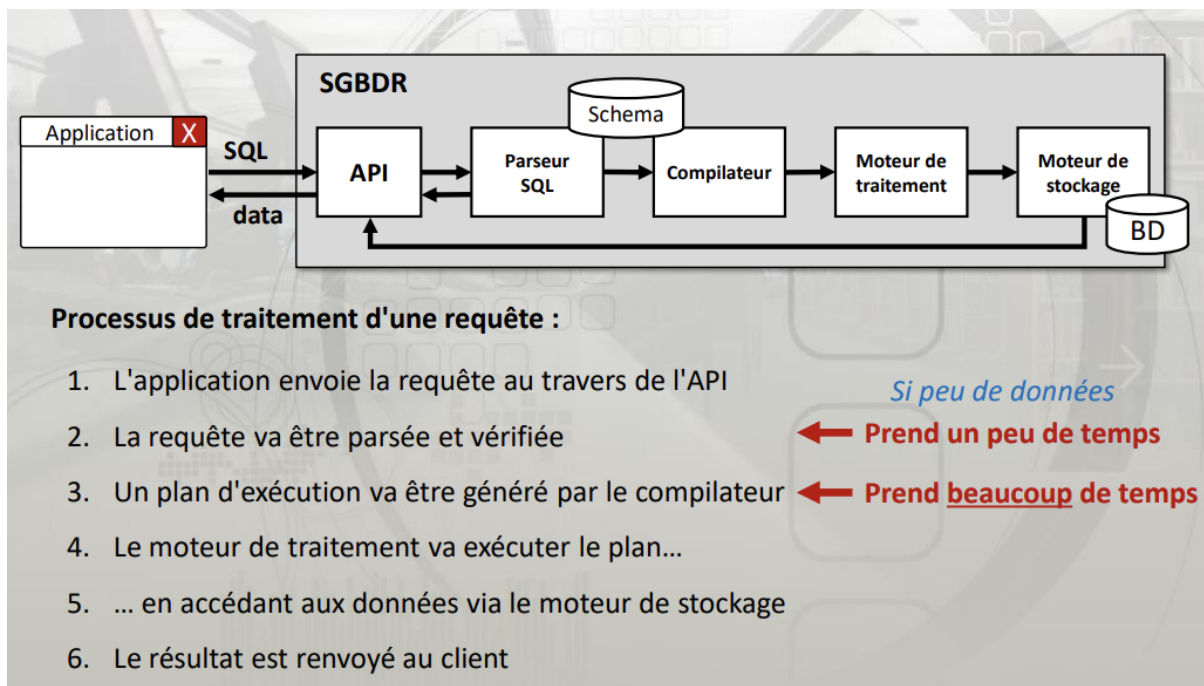
Inconvénients :

- Occupent de l'espace
- Ralentissent les mises à jour (INSERT, UPDATE et DELETE)

Différentes structures d'indexation (B-Arbre, Hash, bitmap)

Requêtes préparées

Architecture interne d'un SGBDR



L'idée première des requêtes préparées :

- Court-circuiter la phase d'analyse du parseur ainsi que toutes les étapes du compilateur

Processus de traitement d'une requête préparée :

Phase 1 :

1. L'application envoie la requête au travers de l'API
2. La requête va être parsée et vérifiée
3. Un plan d'exécution va être généré par le compilateur
4. Sauvegarde du plan et envoi d'un identifiant de requête au client

Phase 2 :

1. Le client envoie l'identifiant de plan accompagné d'éventuels paramètres
2. Le moteur de traitement va exécuter le plan...
3. ... en accédant aux données via le moteur de stockage
4. Le résultat est renvoyé au client

Préparation d'une requête

- `PREPARE test FROM 'SELECT * FROM matable WHERE ID=? AND truc>?';`
Les '?' présents dans la requête représentent des paramètres

Avantages des requêtes préparées

1. Protection contre les injections SQL
2. Accélèrent les scripts qui exécutent une même requête en boucle
3. Peuvent réduire sensiblement la charge réseau pour les scripts du point ci-dessus

Ce genre de script est peu courant dans un système bien conçu.

Inconvénients des requêtes préparées

1. Légère baisse de réactivité
2. Légère augmentation de la charge réseau

Virtuellement 2 interactions client/serveur pour une seule requête

Les vues

Utilité principale :

- Méthode d'abstraction qui permet de rendre l'application agnostique du schéma

Utilité secondaire :

- Offre un mécanisme de confidentialité supplémentaire en permettant de masquer des colonnes à un compte qui n'aurait accès qu'aux vues.

Fonctionnalités souvent méconnue :

- Il est possible de mettre à jour les tables si la requête originelle n'est pas ambiguë
- S'il est possible d'effectuer des mises à jour et que la requête originelle n'est "vraiment" pas ambiguë, il peut également être possible d'effectuer des insertions. (exemple : toutes les colonnes non-nulles et sans valeur par défaut doivent être présentes dans la requête)

VENDEURS
<u>id_vendeur</u>
nom
prenom

RECETTES
<u>id_vendeur</u> (FK)
<u>date_recette</u>
montant

Nous voulons **créer une VUE** pour récupérer le total des recettes par vendeur et par mois. Nous voulons obtenir les colonnes 'année', 'mois', 'nom', 'prénom' et 'recettes' :

```
CREATE VIEW recettes_mensuelles_vendeurs AS  
SELECT YEAR( r.date_recette ) AS 'annee', MONTH( r.date_recette ) AS 'mois',  
        v.id_vendeur, v.nom, v.prenom, SUM( r.montant ) AS 'recettes'  
FROM (recettes AS r ) JOIN (vendeurs AS v) ON (v.id_vendeur=r.id_vendeur)  
GROUP BY annee, mois, r.id_vendeur
```

La vue s'utilise ensuite comme une table :

```
SELECT * FROM recettes_mensuelles_vendeur WHERE annee=2019
```

Trigger → va changer un truc dans un table quand quelque chose se passe

Les déclencheurs peuvent être déclenchés (...) à partir des commandes DML suivantes :

- INSERT
- UPDATE
- DELETE

Le code exécuté par le trigger peut se faire :

- Soit avant ou après un INSERT, UPDATE ou DELETE
- Soit une fois par ligne modifiée, soit une fois par requête SQL

2. On crée le trigger qui devra se déclencher pour chaque insertion dans la table recettes :

DELIMITER |

```
CREATE TRIGGER trigger_insert_recettes
BEFORE INSERT ON recettes
FOR EACH ROW
BEGIN
    INSERT INTO vuemat_recettes (annee, mois, id_vendeur, recettes)
    VALUES (YEAR(NEW.date_recette), MONTH(NEW.date_recette),
            NEW.id_vendeur, NEW.montant)
    ON DUPLICATE KEY
    UPDATE recettes= recettes + NEW.montant;

```

END |

DELIMITER ;

RECETTES
<u>id_vendeur</u> (FK)
<u>date_recette</u>
montant

VUEMAT_RECETTES
<u>annee</u>
<u>mois</u>
<u>id_vendeur</u> (FK)
recettes

3. On crée le trigger qui devra se déclencher pour chaque UPDATE pour la table recettes :

DELIMITER |

```
CREATE TRIGGER trigger_update_recettes
BEFORE UPDATE ON recettes
FOR EACH ROW
BEGIN
    UPDATE vuemat_recettes

    SET recettes= recettes + (NEW.montant – OLD.montant)

    WHERE annee=YEAR(NEW.date_recette)
      AND mois=MONTH(NEW.date_recette)
      AND id_vendeur=NEW.id_vendeur;
END |
DELIMITER ;
```

RECETTES
<u>id_vendeur</u> (FK)
<u>date_recette</u>
montant

VUEMAT_RECETTES
<u>annee</u>
<u>mois</u>
<u>id_vendeur</u> (FK)
recettes

Trigger et vues matérialisées



- **Les vues matérialisées sont des vraies tables et permettent d'accélérer les recherches :**
 - possibilité de créer des index ;
 - données directement accessibles (pas besoin de les calculer dynamiquement) ;
 - contiennent souvent moins de données que les tables sources (pas toujours nécessaire d'indexer).
- **Les déclencheurs :**
 - offrent un mécanisme transparent pour les "codeurs";
 - permettent de maintenir la BD dans un état cohérent.



- **Les vues matérialisées :**
 - occupent de la place;
 - introduisent de la redondance.
- **L'utilisation des déclencheurs à un coût :**

Toute opération d'écriture dans une table pourvue de déclencheurs provoquera un **overhead** lié aux écritures additionnelles et aux éventuels maintiens des index.

Gestion des contraintes d'intégrité référentielle



FOREIGN KEY (`id\_vendeur`) **REFERENCES** `vendeurs`(`id\_vendeur`)
ON DELETE CASCADE ON UPDATE CASCADE

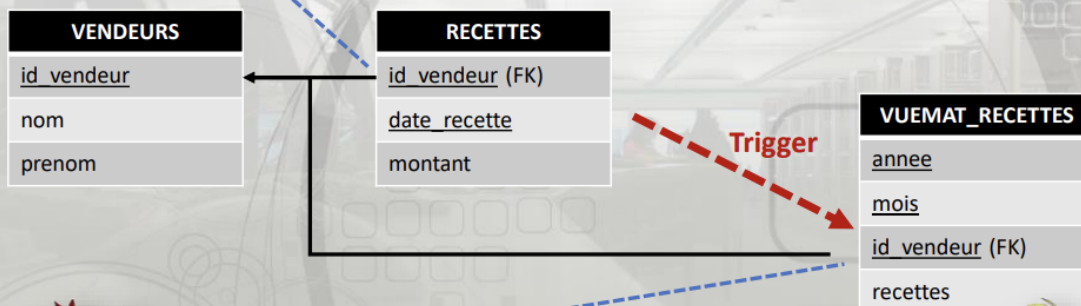
Si on efface un vendeur, alors toutes les entrées de ce vendeur dans la table recettes seront automatiquement supprimées (possible avec InnoDB).

On pourrait s'attendre à ce que la suppression des enregistrements de la table recettes provoque le trigger associé à cette table mais avec MySQL/MariaDB... il n'en est rien.

ATTENTION : avec MySQL/MariaDB, les déclencheurs sont liés aux commandes SQL, pas aux opérations internes du SGBD...

Gestion des contraintes d'intégrité référentielle

FOREIGN KEY (`id\_vendeur`) **REFERENCES** `vendeurs`(`id\_vendeur`)
ON DELETE CASCADE ON UPDATE CASCADE



FOREIGN KEY (`id\_vendeur`)
REFERENCES `vendeurs`(`id\_vendeur`)
ON DELETE CASCADE ON UPDATE CASCADE

Gestion des contraintes d'intégrité référentielle

Solution 4 : Avec MySQL et MariaDB, on se passe des effets de CASCADE pour les clés étrangères et on utilise des triggers un peu partout :



Comme le trigger contient du SQL, en cas de suppression d'un vendeur, le second trigger sera automatiquement exécuté.

Fonction :

Il est possible de créer ses propres fonctions dans le SGBD :

```
CREATE FUNCTION cm2inch(cm FLOAT)
RETURNS FLOAT
BEGIN
    RETURN (cm / 2.54);
END
```

```
mysql> select cm2inch(25.4);
+-----+
| cm2inch(25.4) |
+-----+
|          10 |
+-----+
1 row in set (0.00 sec)
```

Contrairement à une procédure, une fonction doit fournir une sortie (une valeur ou une relation)

Une fonction en peut pas :

- Gérer des transactions
- Faire des mise à jour (INSERT, UPDATE, DELETE)
- Pas de DDL (CREATE, ALTER, TRUNCATE, etc.)
- Pas d'EXEC/CALL (appel à des procédures stockées)

Procédure :

C'est comme une fonction, mais sans les limitations inerrantes.

```
CREATE PROCEDURE nom_de_la_procedure([parametres])  
BEGIN  
...  
END
```

Avec les procédures stockées, les paramètres peuvent être :

- **IN**
- **OUT**
- **INOUT**

Une procédure est appelée avec la commande : **CALL maprocedure('machin');**

Contrairement aux requêtes préparées, les fonctions et procédures stockées ne sont pas stockées en session utilisateur. Elles sont sauvegardées durablement sur le serveur.

Elles peuvent même être déclenchées par un trigger



- **Fournissent un mécanisme d'abstraction du schéma pour les utilisateurs :**

- Code de l'application plus simple
- Pas d'accès direct aux tables (meilleure sécurité)

- **Meilleure réactivité car les procédures stockées sont précompilées**

- **Gains substantiels au niveau de la charge réseau :**

- Requêtes plus courtes
- Moins d'interactions

- **Possible d'appeler des scripts externes**



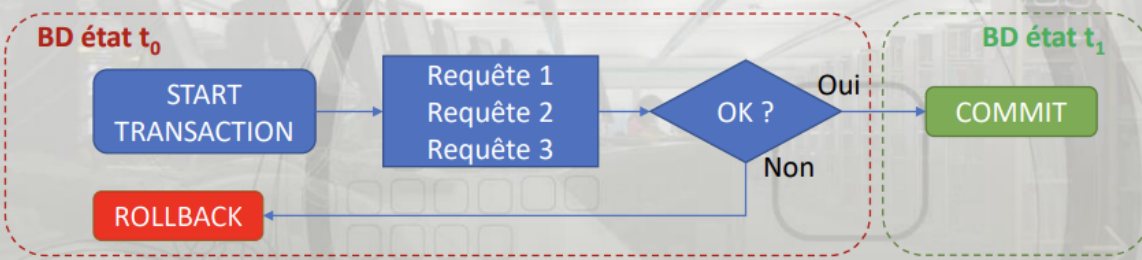
- **Syntaxe plutôt "has been" et fonctionnalités limitées du langage**

- **Elles sont précompilées, ce qui ne protège pas totalement contre les injections SQL si l'implémentation est maladroite.**

- **Nécessite un peu de "skill"**

Transaction

Les transactions permettent de s'assurer qu'un ensemble de requêtes puisse être correctement traité avant de l'exécuter.



OLTP (Online Transaction Processing) :

(traitement transactionnel en ligne) :

*Contexte d'utilisation typique des activités opérationnelles
(e-commerce, production, système de réservation, etc.)*

Un système OLTP doit :

- Pouvoir répondre à un nombre élevé de transactions (ensemble solidaire de requêtes) par minute.
- Doit être performant tant en lecture qu'en écriture.
- Garantir l'ACIDité des transactions.

Gestion des accès concurrent :

- Verrouillage pessimiste : La ressource est accessible à une seule transaction et les autres sont en attente
- Verrouillage optimiste : Les accès peuvent se faire concurremment, le SGBD conserve différentes versions d'une ressource associées à éventuellement un Timestamp

MVCC (Multiversion concurrency control) en optimiste : Chaque version d'une ressource dispose d'un tag (numéro de transaction ou timestamp) /!\ peut être pessimiste aussi

Partitionnement

Sharding : Diviser une table (ou BD) pour la répartir (le plus souvent) entre $\neq$ serveur

Règle de partitionnement peut être faite manuellement ou automatiquement

Le partitionnement horizontal

	Col1	Col2	Col3	Col4
1		Xxx	Aaa	Fff
2		Yyy	Bbb	Ggg
3		Zzz	Ccc	Hhh
4		www	ddd	ljj

	Col1	Col2	Col3	Col4
1		Xxx	Aaa	Fff
2		Yyy	Bbb	Ggg

	Col1	Col2	Col3	Col4
3		Zzz	Ccc	Hhh
4		www	ddd	ljj

Le partitionnement peut se faire :

- de manière automatique et transparente :

```
if ( id%2==1) -> Node 1
else -> Node 2
```
- ou dynamiquement sur base de règles établies au préalable :

```
if ( sexe=='m') -> Node 1
else -> Node 2
```

Avantage Horizontal

- Le partitionnement horizontal offre la possibilité au système d'interroger les fragments de table simultanément par les différents serveurs :
 - gain en rapidité
 - les index sont également fragmentés (gain en rapidité d'écriture)
- Load balancing pour les écritures

Désavantage Horizontal

- Infrastructure plus complexe
- Augmentation de la latence

- Les recherches croisées (jointures) peuvent saturer le canal de communication utilisé entre les serveurs (bottleneck).

Réplication

Nécessite au minimum 2 serveurs : 1 Master et 1 Slave

Utilité première : avoir une copie de la BD

Autre utilité : soutenir de fortes charges

Il existe différents types de réplication :

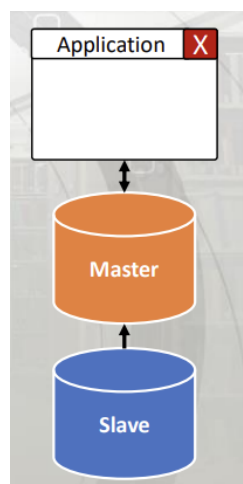
▼ Réplication asynchrone

Le plus simple

Le Master n'attend pas le Slave

Le Master inscrit ses event dans un journal de log, que le slave va check tout les x (lag) temps

Si Master crash → perte des X dernière transactions

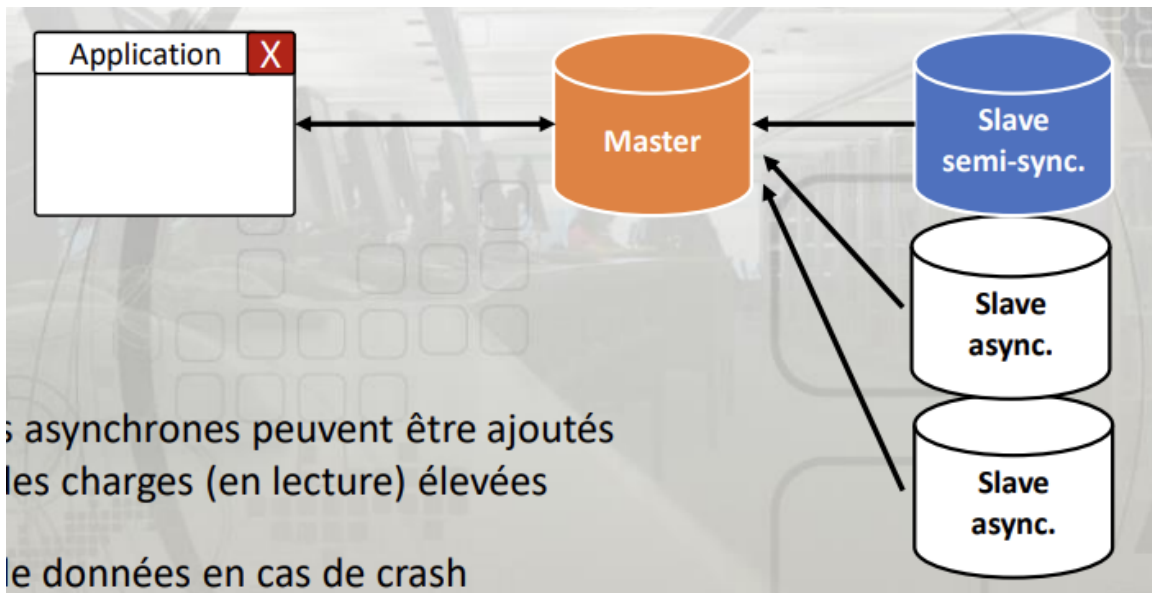


▼ Réplication semi-synchrone

Le maître ne valide pas la transaction tant qu'au moins un esclave ne l'a pas répliqué

Évite les pertes de données en cas de crash

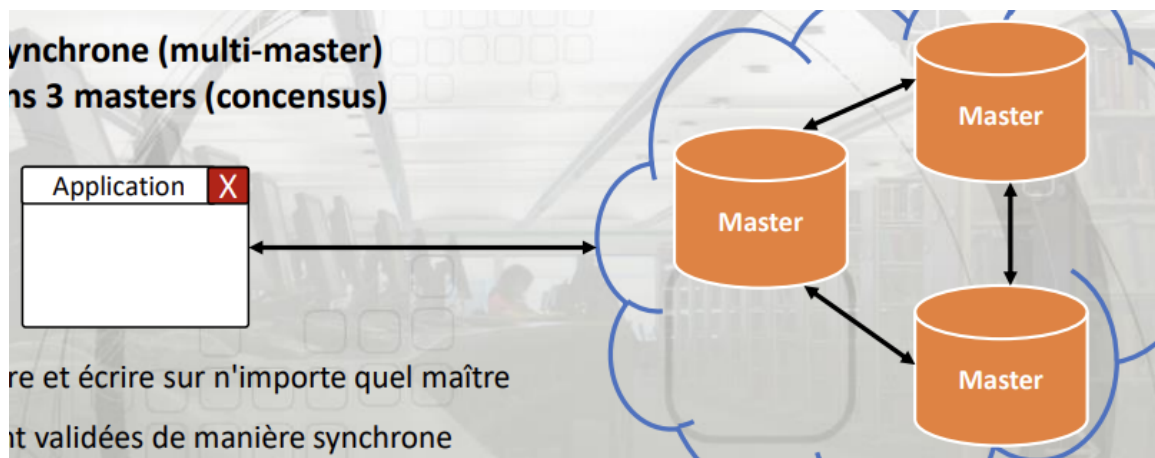
En cas de crash, c'est l'esclave semi-synchrone qui est promu maître



▼ Réplication synchrone

Une réplication synchrone (multi-master) nécessite au moins 3 masters (consensus)

L'application peut lire et écrire sur n'importe quel maître

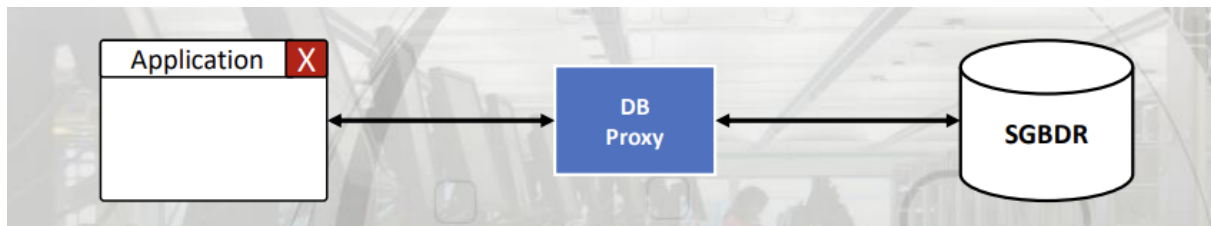


▼ Réplication logique (transactionnelle)

Ce n'est pas la DB mais les tables qui sont répliquées

Un proxy est un logiciel qui gère la connectivité entre 2 parties

L'application cliente est connectée au proxy en pensant que c'est le SGBDR



Technologie sémantique :

- Permet d'effectuer des recherches sur base du sens des valeurs et non sur la valeur des valeurs ...
- Ajoute des nouveaux index et fonctionne en complément de l'indexation FULL-Text

Fuzzy Search (recherche approximative) :

- Trouver des chaînes de caractères qui correspondent à un motif approximatif
- Exemple : entrer "algorithme" et le système trouve "algorithme"

Multi-tenant / multi-base :

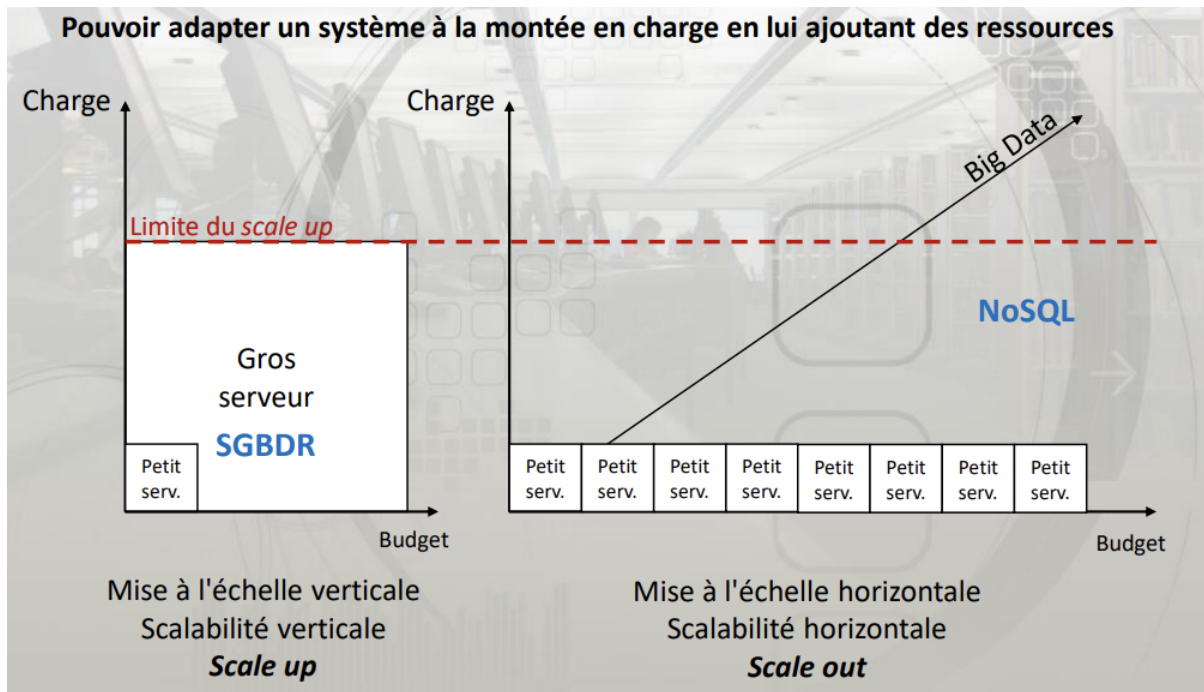
- Permet d'exécuter des requêtes qui impliquent plusieurs bases de données
- Natif dans SQL Server et avec des performances optimisées
- Non natif mais possible avec Oracle(option \$\$\$) et PostgreSQL
- Natif avec MySQL/MariaDB

Les SGBD découpent les données en fragments : pages

Ce sont les pages qui sont la plus petite unité de la BD qui est lue/écrite sur le disque

Big data > 1To

Scalabilité :



▼ EXO SQL

1. **Sélectionnez le nom de tous les plats dont le prix est strictement inférieur à 5 :**
 - `SELECT nom FROM plats WHERE prix < 5;`
2. **Modifiez le l'id_type du King-Bacon, au lieu de NULL, elle devient 2 :**
 - `UPDATE plats SET id_type=2 WHERE id=4;`
3. **Ajoutez un enregistrement à la table plat un enregistrement qui a comme valeur : id_type=1, nom = américain, prix = 2.5 (la clé primaire est auto incrémentée) :**
 - `INSERT INTO plats(id_type,nom,prix) VALUE(1,'américain',2.5);`
4. **Donner le prix moyen des plats de type sandwich :**
 - `SELECT AVG(prix) FROM plats GROUP BY id_type WHERE id_type=1;`
 - `SELECT AVG(p.prix) FROM plats as p JOIN type_plat as t ON p.id_type=t.id_type GROUP BY t.id_type WHERE t.nom_type = 'sandwich'`
 - `SELECT AVG(p.prix) FROM plats as p NATURAL JOIN type_plat as t GROUP BY t.id_type WHERE t.nom_type='sandwich'`

5. **Sélectionnez tous les noms des types de plat ainsi que leurs plats associés même si un nom de type de plat n'est pas associé à un plat :**

- `SELECT t.nom_type, p.nom FROM type_plat as t LEFT JOIN plats as p ON p.id_type=t.id_type`

6. **Affichez le type de plat ainsi que la valeur du stock de ces types qui ont plus de 10 éléments en stock et utiliser un alias pour nommer le résultat de la colonne valeur :**

- `SELECT t.nom_type, SUM(p.stock*p.prix) AS valeur FROM plats AS p NATURAL JOIN type_plat as t GROUP BY p.id_type HAVING SUM(p.stock)>10`

Commit/rollback → Contrôle la bonne exécution des transactions

exo 1)

```
SELECT ville.nom, ville.population2012 FROM VILLE ORDERBY vil
```

```
SELECT ville.nom, ville.superficie FROM VILLE ORDER BY (j'ai
```

```
SELECT departement_nom FROM departement WHERE departement_cod  
le code commence par 97 et un caractère quelconque (le %)
```

```
SELECT v.ville.nom, d.departement.departement_nom FROM villes.  
JOIN departement AS d ON v.ville_departement = d.departement_  
v.ville_population_2012 DESC LIMIT 10
```

```
SELECT d.departement_nom, d.departement_code, COUNT(v.ville_i  
FROM departement AS d JOIN villes_france_free AS  
v on d.departement_code = v.ville_departement GROUP BY v.vil  
ORDER BY nombre_commune DESC
```